

Part A: System Architecture Design

1. Which components communicate using WebSocket?

The sensors and the server talk to each other using WebSocket, and the server also uses it to send updates to the dashboard. So basically the whole chain from the sensor to what you see on screen is connected through WebSocket.

2. Where does data processing or model inference occur?

The processing happens on the server. When the sensor data arrives, the server runs it through the model to check if the crops are okay or not, then sends the result straight to the dashboard.

3. Why is WebSocket preferred over REST for live monitoring?

With REST, the dashboard has to keep asking the server if there is new data, even when there is nothing new. WebSocket skips all that by keeping the connection open so the server can just send updates whenever something changes. For a live monitoring system that is a lot more practical.

Part B: Data Flow Description

1. Sensor data generation

The sensors on the farm are constantly measuring things like temperature and humidity and sending those readings out at regular intervals.

2. WebSocket connection establishment

The device connected to the sensors sends a request to the server to open a WebSocket connection. Once the server agrees, the connection stays open and both sides can communicate freely.

3. Data transmission

The sensor readings are sent to the server continuously through that open connection. No need to reconnect every time, it just keeps flowing.

4. Model analysis

Once the server receives the data, it feeds it to the model which checks the values and decides whether the crops are Healthy, Warning, or Critical.

5. Real time update to the dashboard

The server immediately sends the result to the dashboard through the same connection. The farmer sees the updated status right away without refreshing anything.

Part C: Model Integration

1. What type of data will the model receive?

The model gets the sensor readings like temperature, humidity, and crop health values. These are just numbers that the model uses to figure out what condition the crops are currently in.

2. How will WebSocket deliver model predictions to the client?

After the model gives a result, the server wraps it up and sends it to the dashboard through the open WebSocket connection. The dashboard picks it up and shows it instantly.

3. What happens when abnormal data is detected?

If the model flags something as Warning or Critical, the server sends an alert to the dashboard right away. The farmer gets notified immediately so they can do something about it before it gets worse.

Part D: Reflection

What I understand about WebSocket is that it solves a problem that HTTP polling creates. With polling, every client keeps asking the server for updates over and over even when nothing has changed, which is wasteful especially when you have a lot of devices connected at the same time. WebSocket just keeps one connection open and only sends data when there is actually something new to send. That alone makes a big difference in how much load the server has to handle. In an IoT setup where you might have dozens or hundreds of sensors running, that kind of efficiency really matters. The system stays fast and responsive without being overwhelmed by constant requests. So overall I think WebSocket just makes more sense for anything that needs to be monitored in real time.